SoDa
ODISSEI Social Data Science Team

Future-proof your research workflow with Git

An introductory workshop

Taymara Abreu, Qixiang Fang, Max Maes, Vlad Mutruc

ODISSEI Early Career Day

4 November 2026 · 13:30–15:45

edu.nl/qe4gw

We help social scientists with data-intensive & computational research

Our goal is to enhance the evidence base and impact of social science by bringing the added value of new data sources and new data analysis techniques into social research in the Netherlands.

Who we are

Core team

Computer and data scientists at postdoc or assistant professor level, with a social sciences background.

Fellows

Early career social scientists working on projects aligned with ODISSEI and SoDa's mission.

Student and research assistants

Support SoDa projects and activities.

Associate members

Liaisons, including research software engineers from eScience, SCP etc.



Fellowship programme

- **2–3 fellows per year**
- For PhD researchers, postdocs and assistant professors
- Propose your own social science research project
- **3–6 months full-time, with salary paid**
- Join the **SoDa team** at Utrecht University's Methodology & Statistics department
- Receive mentoring from a senior team member

Next application deadline:
7 December

Agenda

Time	Activity
13:30–13:40	Git, GitHub and research files
13:40–14:00	Repository setup and cloning
14:00–14:25	Changes, commits, history, push/pull
14:25–14:35	Break
14:35–14:55	Branches
14:55–15:30	Issues, pull requests and peer review
15:30–15:45	Git and AI-assisted research

“Can you send me the previous version?”

Last month, you sent your supervisor some analysis results.

Since then, you’ve excluded some respondents and changed some variables.

Your supervisor asks:

> **“Could we go back to the original model and compare the results?”**

You open your project folder. **Which script was the one you used?**

Finding the script you sent last month

Bad

```
research-project/
  data/
  results/
  README.md
  analysis.R
```

Better?

```
research-project/
  data/
  results/
  README.md
  analysis.R
  analysis_final.R
  analysis_final_v2.R
  analysis_Lisa.R
  analysis_new.R
  analysis_revised.R
  older_versions/
```

Which version did you use for the analysis you sent your supervisor?

Keeping track of a research project

A research project changes as you revise the analysis and respond to feedback.

Git helps you:

- Record **what changed and why**
- Return to an earlier version of your files
- Combine changes from collaborators
- Maintain control of AI-generated code/text

You keep working in the same files. **Git records the versions you choose to save.**

Finding the script you sent last month

Without version control

```
research-project/
  data/
  results/
  README.md
  analysis.R
  analysis_final.R
  analysis_final_v2.R
  analysis_Lisa.R
  analysis_new.R
  analysis_revised.R
  older_versions/
```

With Git

```
research-project/
  data/
  results/
  README.md
  analysis.R
```

History recorded in Git for analysis.R

Nov. 3: “Restrict sample to employed respondents”

Nov. 2: “Add education as a control”

Nov. 1: “Remove outliers”

Oct. 10: “Add regression model”

Changes | History

Select Branch to Compare...

docs: link preliminary current app assessment
Qixiang Fang • 21 hours ago

docs: assess current app against FAIRR checklists
Qixiang Fang • 21 hours ago

docs: retain current app probe results
Qixiang Fang • 21 hours ago

docs: add repeatable engine and export evidence probe
Qixiang Fang • 21 hours ago

docs: update checklist source mapping guidance
Qixiang Fang • yesterday

docs: index operational assessment checklists
Qixiang Fang • yesterday

docs: add operational FAIRR capability checklist
Qixiang Fang • yesterday

docs: add operational FAIR4RS software checklist
Qixiang Fang • yesterday

docs: link assessment framework from README
Qixiang Fang • yesterday

docs: document FAIRR assessment sources
Qixiang Fang • yesterday

docs: define FAIRR assessment framework
Qixiang Fang • yesterday

fix: restrictions didnt allow to load the app + ruff codestlye fixes
Vlad-Andres • 2 days ago

Merge pull request #94 from sodascience/security/hardening
V. I. Mutruc • 2 days ago

feat: enhance security and integrity checks by adding SHA-256 validation ...
Vlad-Andres • 2 days ago

hotfix: fix security vulnerabilities by sanitizing markdown rendering in UI c...

fix: restrictions didnt allow to load the app + ruff codestlye fixes

Vlad-Andres 0ab0e31 +10 -12

11 changed files

- client/ui/compon.../datasets.py
- client/ui/compon.../error_log.py
- client/ui/compone.../exports.py
- client/ui/c.../projects_modal.py
- client.../prompt_result_table.py
- client/ui/compone.../prompts.py
- client/ui/c.../settings_modal.py
- client/ui/comp.../split_picker.py
- client/ui/mount.py
- client/ui/workspace.py
- index.html

client/ui/components/datasets.py

```
@@ -9,7 +9,7 @@ from panel.viewable import Viewer
 9  9  from components.dataset_files import ProjectFilesExplorer
10 10  from components.dataset_preview import DatasetPreview
11 11  from components.dataset_subset import DatasetSubset
12    - from shared import md_text, notifications, report_error, reporting
   12 + from shared import md_text, report_error, reporting
13 13  from store import Store
14 14  from theme import GUTTER, NAV_HEIGHT, card, card_title, section_label
15 15  from workspace import Workspace
```

Workshop goals

By the end, you can:

- **Track** changes to a research project.
- **Recover** an earlier version when something goes wrong.
- **Synchronize** work between your computer and GitHub.
- **Work in branches** without disturbing the main version.
- **Propose, review, and merge** a colleague's changes.
- **Understand** how to further integrate Git into your (AI-assisted) research workflow.

> You do not need to write or run code to learn any of these.

Three different tools

Git

The foundation: version control

History · Commits · Branches · Merges

⇅

Use Git to work with repositories

GitHub.com

Website: share · review · discuss

GitHub Desktop

App on your laptop: commit · push · pull

Other clients work the same way, e.g. GitLab, VS Code, RStudio, Overleaf

A change in three steps

YOUR COMPUTER

1 **Edit**

Change and save a file

2 **Commit**

Record a snapshot with a message

GITHUB

3 **Push**

Share your commits

Pull works the other way: bring changes from GitHub to your computer

FAIRR-LLM

Private

Watch 0 Fork 0 Star 0

main 8 Branches 0 Tags Go to file T Code

Vlad-Andres Merge pull request #39 from sodascience/docs/... c825e0b · 2 weeks ago 29 Commits

.github/workflows	fix: small reformat of the deploy file	last month
docs/architecture	docs: update architecture diagrams to use M...	last month
.env.example	Encapsulation of the changes under the doc...	3 months ago
.gitignore	Update gitignore	3 months ago
README.md	Add README	3 months ago

README

FAIRR-LLM-Playground Design Document

About

Open-source platform that enables researchers to conduct FAIR and reproducible LLM-based annotations without requiring extensive programming skills.

sodascience.github.io/FAIRR-LLM/

annotation-tool fairness llm reproducibility

Readme

Activity

Custom properties

0 stars

0 watching

0 forks

Current Repository
workshop-future-proof-research-workflows-with-git

Current Branch
main

Fetch origin
Last fetched 1 minute ago

Changes | History

Filter

0 changed files

Summary (required)

Description

Commit to **main**

No local changes

There are no uncommitted changes in this repository. Here are some friendly suggestions for what to do next.

Open the repository in your external editor
Select your editor in Settings
Repository menu or ⌘ ⇧ A

Open in Visual Studio Code

View the files of your repository in Finder
Repository menu or ⌘ ⇧ F

Show in Finder

Open the repository page on GitHub in your browser
Repository menu or ⌘ ⇧ G

View on GitHub

Exercise 1 · Your private repository (5 min)

1. Sign in at **GitHub.com** and choose **New repository**.
2. Name it `research-git-practice`.
3. Choose **Private** and **Add a README file**.
4. Click **Create repository**.
5. Check the **Private** label and open `README.md`.

Goal: You own a private repository on GitHub.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

1 General

Owner *

fqixiang /

Repository name *

research-git-practice

research-git-practice is available.

Great repository names are short and memorable. How about redesigned-fiesta?

Description

My first GitHub repository.

27 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Private

Start with a template

Templates pre-configure your repository with files.

No template

Add README

READMEs can be used as longer descriptions. About READMEs

On

main 1 Branch 0 Tags Go to file T Code

fqixiang Initial commit 2331e05 · 1 minute ago 1 Commit

README.md	Initial commit	1 minute ago

README

research-git-practice

My first GitHub repository.

About

My first GitHub repository.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

A repository is more than a folder

A **repository** contains:

- Project files: code, documentation, configuration.
- A **history of commits**: what changed, when, and by whom.
- One or more **branches**: different lines of development.

Typical folder structure for a research project

```
├── .git/               # Local Git history and settings (Git manages this)
├── .github/            # Optional GitHub configurations
├── data/
│   ├── raw/            # Original data (read-only)
│   └── processed/      # Data produced by scripts
├── notebooks/          # Jupyter, R Markdown or Quarto notebooks
├── src/                # Source code and scripts
├── .gitignore          # Rules for excluding untracked files
├── LICENSE.txt         # Project license
└── README.md           # Project overview and documentation
```

Dot-prefixed names may be hidden in your file manager.

Exercise 2 · Clone it locally (5 min)

1. On GitHub, open **your** new repository.
2. Click **Code → Open with GitHub Desktop**.
3. Choose where to save the local repository.
4. Click **Clone**.
5. Open the cloned folder in your file manager.

Check: Can you find `README.md` on your computer?

Current Repository
research-git-practice

Current Branch
main

Fetch origin
Never fetched

Changes | History

Filter

0 changed files

Summary (required)

Description

Commit to **main**

No local changes

There are no uncommitted changes in this repository. Here are some friendly suggestions for what to do next.

Open the repository in your external editor
Select your editor in Settings
Repository menu or ⌘ ⇧ A

Open in Visual Studio Code

View the files of your repository in Finder
Repository menu or ⌘ ⇧ F

Show in Finder

Open the repository page on GitHub in your browser
Repository menu or ⌘ ⇧ G

View on GitHub



README.md

Two places

On GitHub.com: your project exists online.

On your laptop: the same files exist in a Git repository.

In GitHub Desktop: you can see your local repository.

Three stages of a change

1. **Edit and save** a file → changed working copy
2. **Commit** in GitHub Desktop → recorded local snapshot with a message

> A commit should describe a coherent change, not just “stuff”.

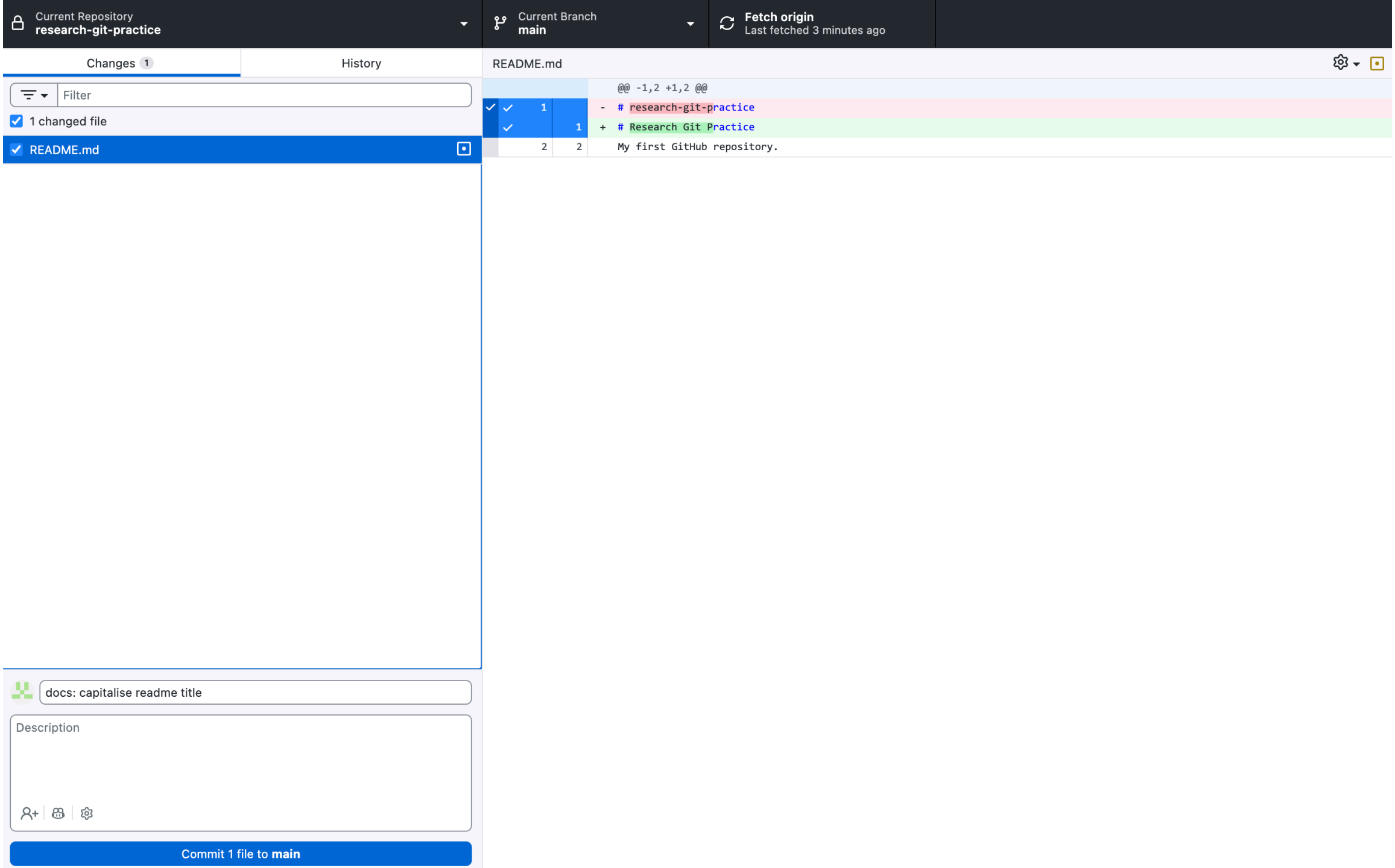
3. **Push** in GitHub Desktop → available on GitHub

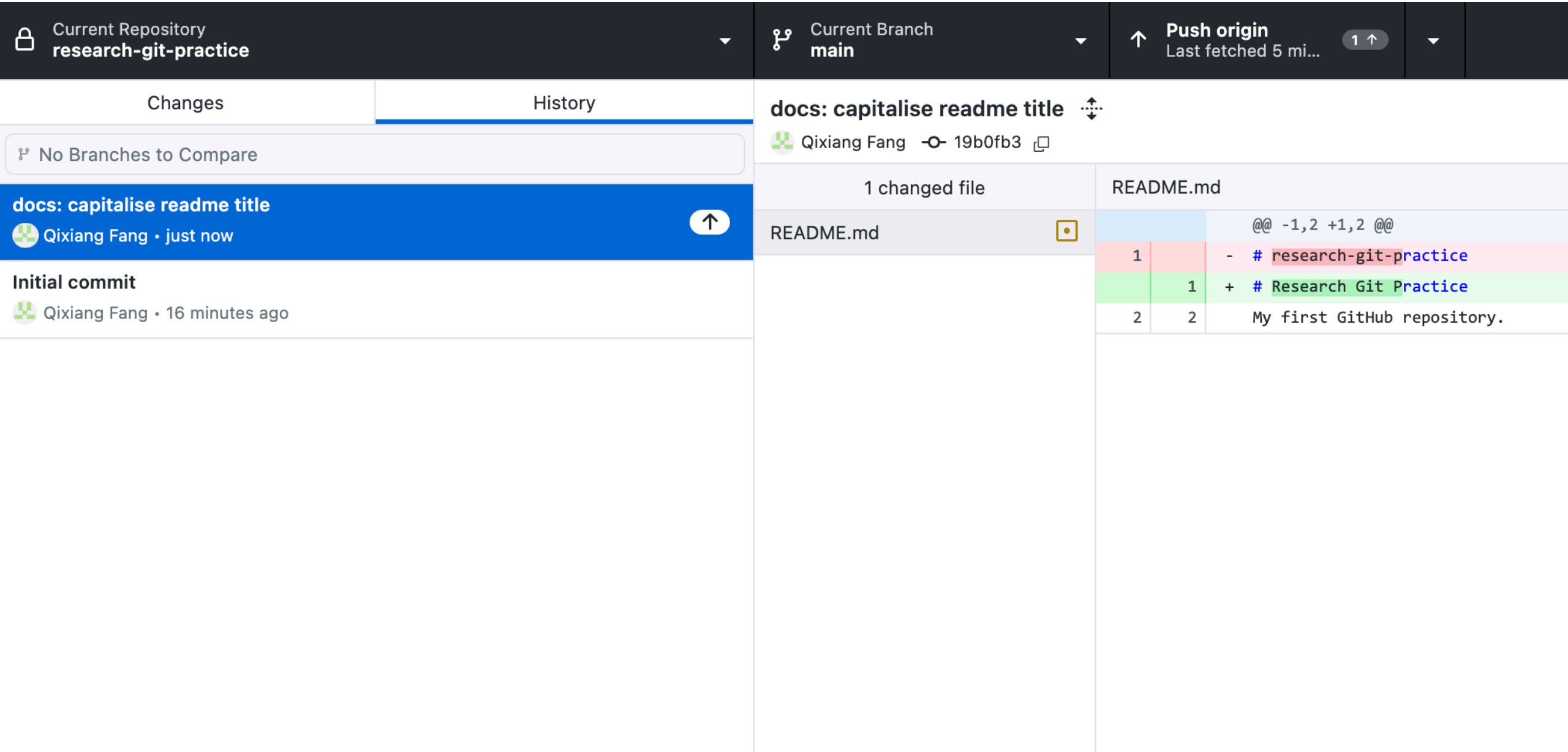
Read the diff before committing

GitHub Desktop → Changes shows:

- **Green / +:** added lines.
- **Red / −:** removed lines.
- File names: which files changed.

> Good commit message: "Document missing-data handling"
> Weak commit message: "updates"


Current Repository
research-git-practice
Current Branch
main
Fetch origin
Last fetched 3 minutes ago
Changes 1
History
Filter
1 changed file
README.md
README.md
@@ -1,2 +1,2 @@
1
- # research-git-practice
1
+ # Research Git Practice
2
2
My first GitHub repository.
docs: capitalise readme title
Description
Commit 1 file to main


Current Repository
research-git-practice
Current Branch
main
Push origin
Last fetched 5 mi...
1
Changes
History
No Branches to Compare
docs: capitalise readme title
Qixiang Fang • just now
Initial commit
Qixiang Fang • 16 minutes ago
docs: capitalise readme title
Qixiang Fang
19b0fb3
1 changed file
README.md
README.md
@@ -1,2 +1,2 @@
1
- # research-git-practice
1
+ # Research Git Practice
2
2
My first GitHub repository.

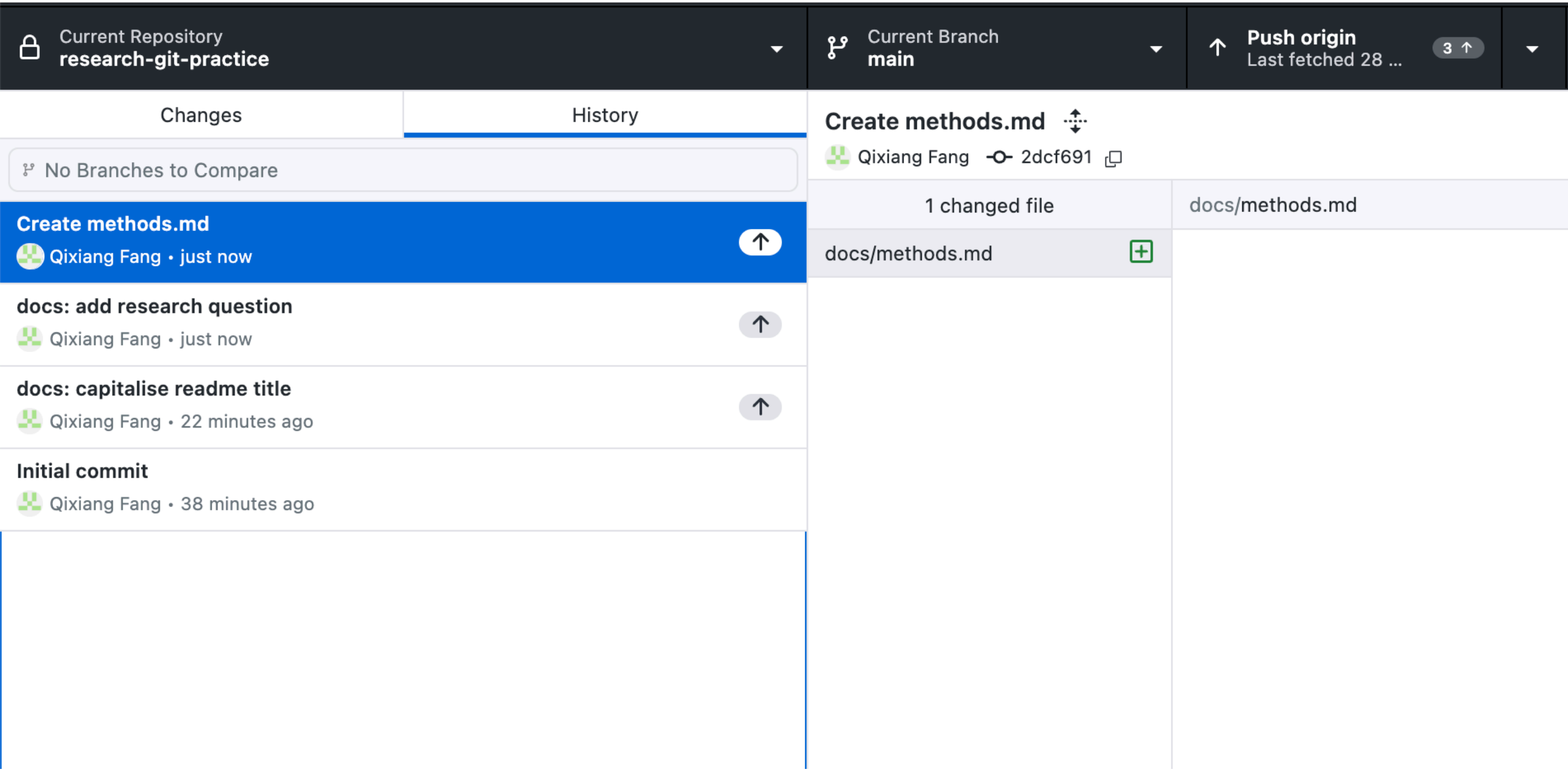
Exercise 3 · Make your first commits (10 min)

1. Open `README.md` in a **plain-text editor**.
2. Add `## Research question` and one research question like:

> What factors influence the number of citations a research publication receives?

3. Create a new file `docs/methods.md`; save.
4. Return to GitHub Desktop → **Changes**.
5. Select README; inspect the diff and describe the change.
6. Click **Commit to main**.
7. Select methods.md; inspect the diff and describe the change.
8. Click **Commit to main**.

Goal: Two new commits in your local history.


Current Repository
research-git-practice
Current Branch
main
Push origin
Last fetched 28 ...
3
Changes
History
No Branches to Compare
Create methods.md
Qixiang Fang • just now
docs: add research question
Qixiang Fang • just now
docs: capitalise readme title
Qixiang Fang • 22 minutes ago
Initial commit
Qixiang Fang • 38 minutes ago
Create methods.md
Qixiang Fang
2dcf691
1 changed file
docs/methods.md
docs/methods.md

Working with data files

SORTD = SoDa Open Research Teaching Datasets: OpenAlex-derived teaching data for exploring research publications.



1. Create a data folder in your cloned repository `/data`
2. Download the zip file `soda-openalex-teaching-1.0.0.zip` to `/data`

Current Repository
research-git-practice

Current Branch
main

Push origin
Last fetched 2 mi...
3 ↑

Changes 1 | History

Filter

1 changed file

data/soda-openalex-teaching-1.0.0.zip

data/soda-openalex-teaching-1.0.0.zip

This binary file has changed.

Open file in external program.

Create soda-openalex-teaching-1.0.0.zip

Description

Commit 1 file to **main**

Committed 7 minutes ago
Create methods.md

Undo

Plain text and binary files

Plain text	Binary formats
`.md`, `.R`, `.py`, `.csv`, `.tex`	`.docx`, `.xlsx`, `.pdf`, images
Git can show changes line by line	Usually no useful line-by-line diff
Easier to review and merge	Often need the original application

Git can store both. **Readable diffs make collaboration easier.**

Do we want to commit the zip file? Why (not)?

What should be committed?

Usually yes

- Analysis scripts and notebooks (when appropriate)
- README and methods documentation
- Small configuration files and environment definitions

Usually no

- Confidential or personal research data
- API keys, passwords, credentials
- Large generated datasets or temporary output files

> **Important:** Deleting something in a later commit does not reliably erase it from Git history.

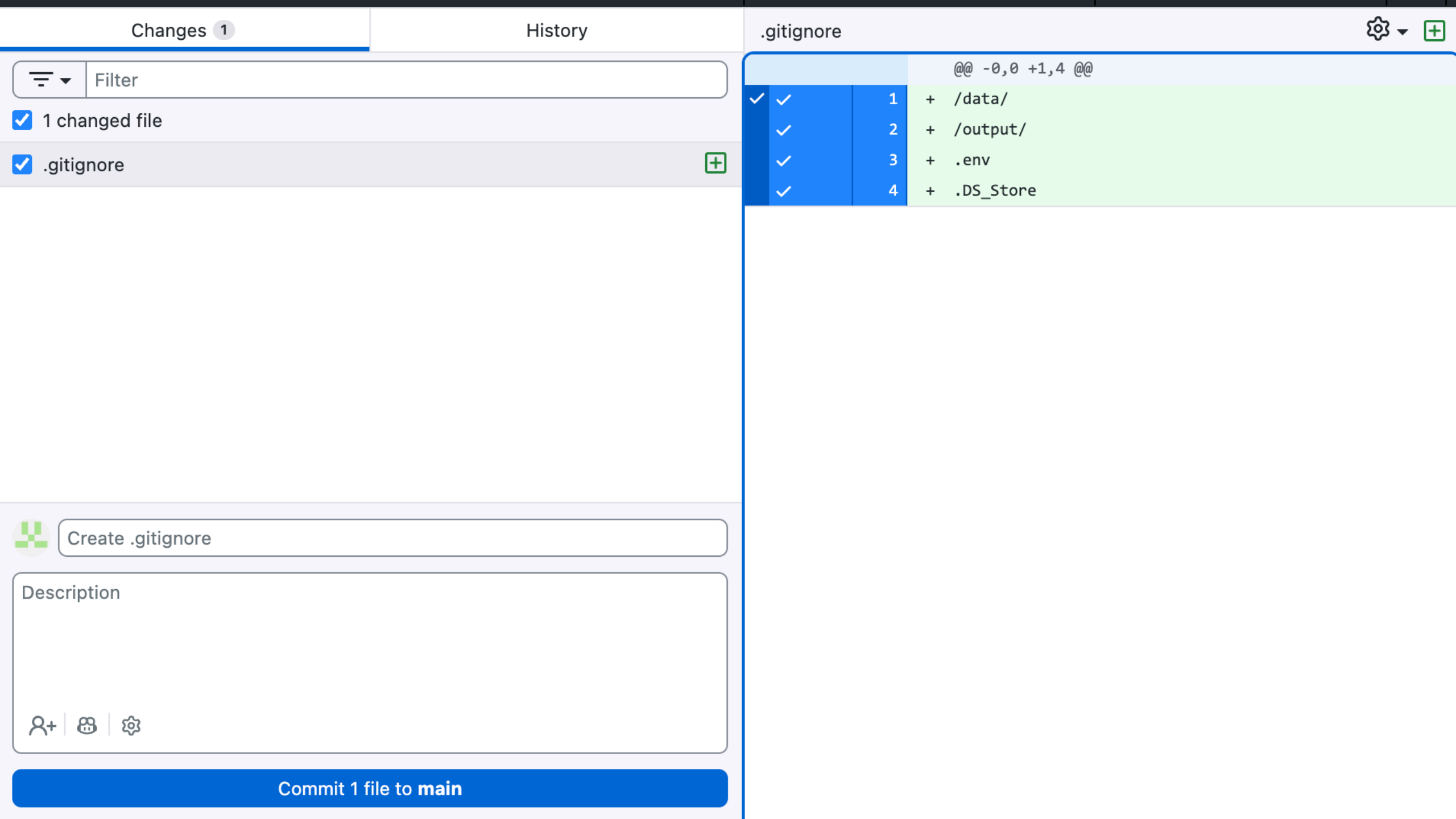
.gitignore

A text file that tells git which files/folders to ignore.

Thou shall not commit these files

1. In your cloned folder, create `.gitignore`.
2. Put these lines in `.gitignore`:

```
/data/
/output/
.env
.DS_Store
```


Changes 1
History
Filter
1 changed file
.gitignore
Create .gitignore
Description
Commit 1 file to main
.gitignore
@@ -0,0 +1,4 @@
1 + /data/
2 + /output/
3 + .env
4 + .DS_Store

Exercise 4 · Publish (5 min)

1. In GitHub Desktop, click **Push origin**.
2. Open the repository on GitHub.com.
3. Confirm the new README text and two new files are there.
4. Confirm the new commit history also appears.

research-git-practice

Private

Watch 0 | Fork 0 | Star 0

main | 1 Branch | 0 Tags | Go to file T | Code

fqixiang Create .gitignore — 559ecbf · 2 minutes ago — 5 Commits

docs	Create methods.md	18 minutes ago
.gitignore	Create .gitignore	2 minutes ago
README.md	docs: add research question	19 minutes ago

README

Research Git Practice

My first GitHub repository.

Research question

What factors influence the number of citations a research publication receives?

About

My first GitHub repository.

Readme
Activity
0 stars
0 watching
0 forks

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors 1

fqixiang Qixiang Fang

Commits

main

All users

All time

Commits on Oct 29, 2026

Create .gitignore
fqixiang committed 4 minutes ago
559ecbf

Create methods.md
fqixiang committed 20 minutes ago
2dcf691

docs: add research question
fqixiang committed 20 minutes ago
014c337

docs: capitalise readme title
fqixiang committed 43 minutes ago
19b0fb3

Initial commit
fqixiang authored 1 hour ago
Verified
2331e05

Exercise 5 · A remote edit and a pull (5 min)

1. On GitHub.com, open `README.md` and click the pencil.
2. Add `Workshop practice: 4 November 2026`.
3. Choose **Commit changes** directly to `main` for this solo exercise.
4. In Desktop, click **Fetch origin**, then **Pull origin**.
5. Open the local README and find the new line.

Check: The browser commit now appears in Desktop's History.

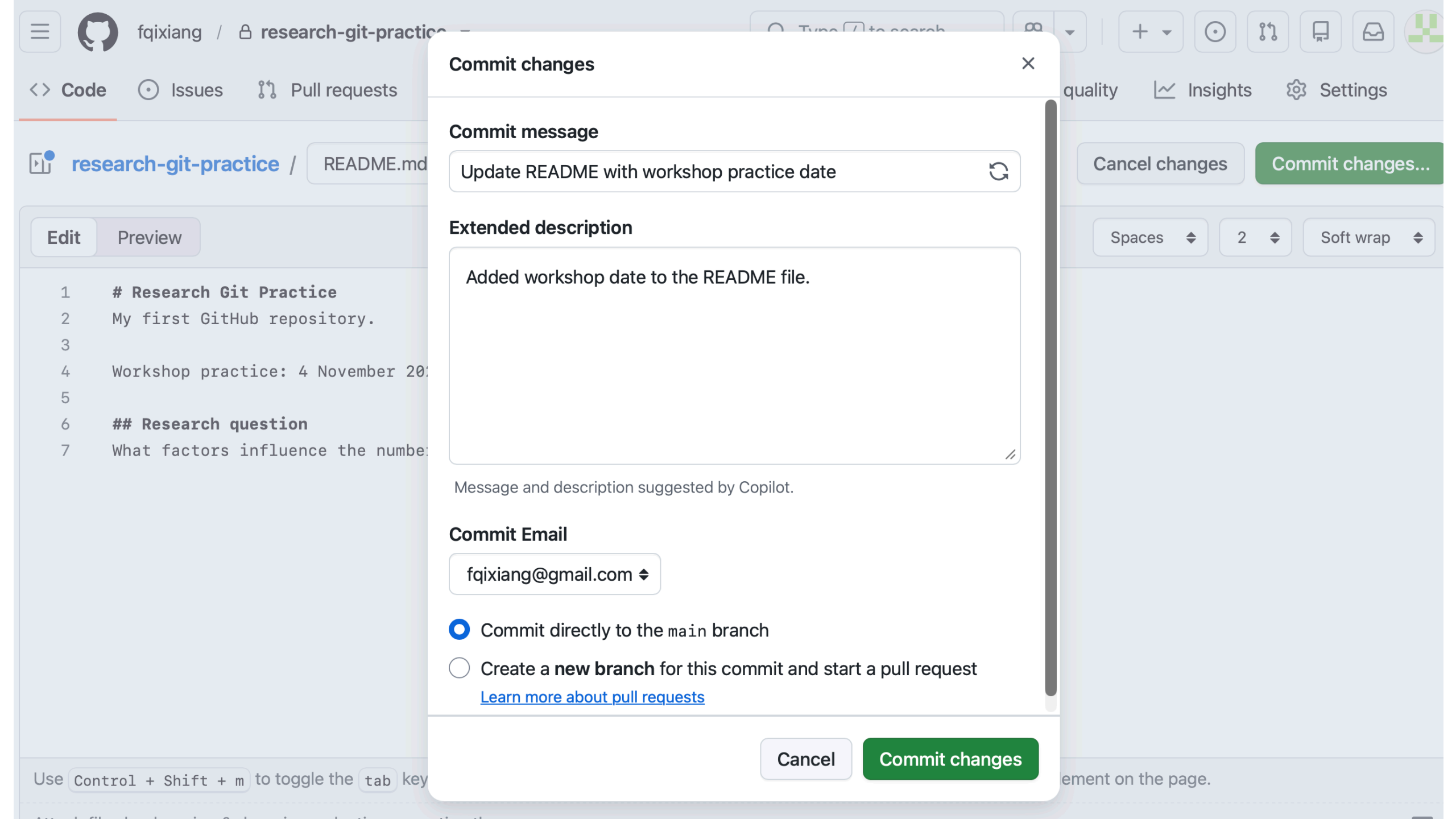
research-git-practice / README.md in main

Cancel changes | Commit changes...

Edit | Preview

Spaces 2 Soft wrap

```
1  # Research Git Practice
2  My first GitHub repository.
3
4  Workshop practice: 4 November 2026
5
6  ## Research question
7  What factors influence the number of citations a research publication receives?
```


fqixiang / research-git-practice
Code
Issues
Pull requests
quality
Insights
Settings
research-git-practice / README.md
Cancel changes
Commit changes...
Edit
Preview
Spaces
2
Soft wrap
1 # Research Git Practice
2 My first GitHub repository.
3
4 Workshop practice: 4 November 20
5
6 ## Research question
7 What factors influence the numbe
Use Control + Shift + m to toggle the tab key
ement on the page.
Commit changes
Commit message
Update README with workshop practice date
Extended description
Added workshop date to the README file.
Message and description suggested by Copilot.
Commit Email
fqixiang@gmail.com
Commit directly to the main branch
Create a new branch for this commit and start a pull request
Learn more about pull requests
Cancel
Commit changes

Current Repository
research-git-practice

Current Branch
main

Pull origin
Last fetched just ...
1

Changes | History

Filter

0 changed files

Summary (required)

Description

Commit to **main**

No local changes

There are no uncommitted changes in this repository. Here are some friendly suggestions for what to do next.

Pull 1 commit from the origin remote
The current branch (`main`) has a commit on GitHub that does not exist on your machine.

Always available in the toolbar when there are remote changes or ⌘ ⇧ P

Pull origin

Open the repository in your external editor
Select your editor in Settings

Repository menu or ⌘ ⇧ A

Open in Visual Studio Code

View the files of your repository in Finder
Repository menu or ⌘ ⇧ F

Show in Finder

Current Repository
research-git-practice

Current Branch
main

Fetch origin
Last fetched 1 minute ago

Changes | History

No Branches to Compare

Update README with workshop practice date
Qixiang Fang • 2 minutes ago

Create .gitignore
Qixiang Fang • 15 minutes ago

Create methods.md
Qixiang Fang • 32 minutes ago

docs: add research question
Qixiang Fang • 32 minutes ago

docs: capitalise readme title
Qixiang Fang • 1 hour ago

Initial commit
Qixiang Fang • 1 hour ago

Update README with workshop practice date

Added workshop date to the README file.

Qixiang Fang c361756 +4 -2

1 changed file

README.md

README.md

```
@@ -1,5 +1,7 @@
1 1   # Research Git Practice
2 -   My first GitHub repository.
  2 + My first GitHub repository.
  3 +
  4 + Workshop practice: 4 November 2026
3 5
4 6   ## Research question
5 -   What factors influence the number o
      f citations a research publication
      receives? No newline at end of f
      ile
  7 + What factors influence the number o
      f citations a research publication
      receives?
```

Undoing changes

- **Discard uncommitted edit:**
  - Gone is gone
  - Discard selected changes only if you truly don't need them.
- **Revert previously committed change:**
  - Use **History → Revert Changes in Commit** to make a new commit that reverses it.

Quiz · Which action?

1. Your edit is saved but absent from History. What comes next?
2. Your commit is local. How do you send it to GitHub?
3. Your colleague made some changes online. How do you update locally?
4. A committed change was wrong. How can you reverse it visibly?

Quiz · Answers

1. **Review the diff and commit.**
2. **Push.**
3. **Fetch, then pull.**
4. **Revert the commit**, then push the new commit.

Break · 14:25–14:35

Next: Collaboration workflows with branches, issues, and pull requests.

Current Repository
research-git-practice

Current Branch
main

Fetch origin
Last fetched 1 minute ago

Changes | History

No Branches to Compare

Update README with workshop practice date
Qixiang Fang • 2 minutes ago

Create .gitignore
Qixiang Fang • 15 minutes ago

Create methods.md
Qixiang Fang • 32 minutes ago

docs: add research question
Qixiang Fang • 32 minutes ago

docs: capitalise readme title
Qixiang Fang • 1 hour ago

Initial commit
Qixiang Fang • 1 hour ago

Update README with workshop practice date

Added workshop date to the README file.

Qixiang Fang c361756 +4 -2

1 changed file

README.md

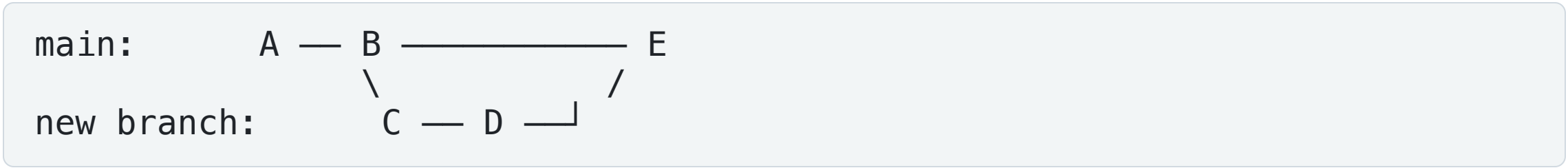
README.md

```
@@ -1,5 +1,7 @@
1 1   # Research Git Practice
2 -   My first GitHub repository.
  2 + My first GitHub repository.
  3 +
  4 + Workshop practice: 4 November 2026
3 5
4 6   ## Research question
5 -   What factors influence the number o
      f citations a research publication
      receives? No newline at end of f
      ile
  7 + What factors influence the number o
      f citations a research publication
      receives?
```

What is a branch?

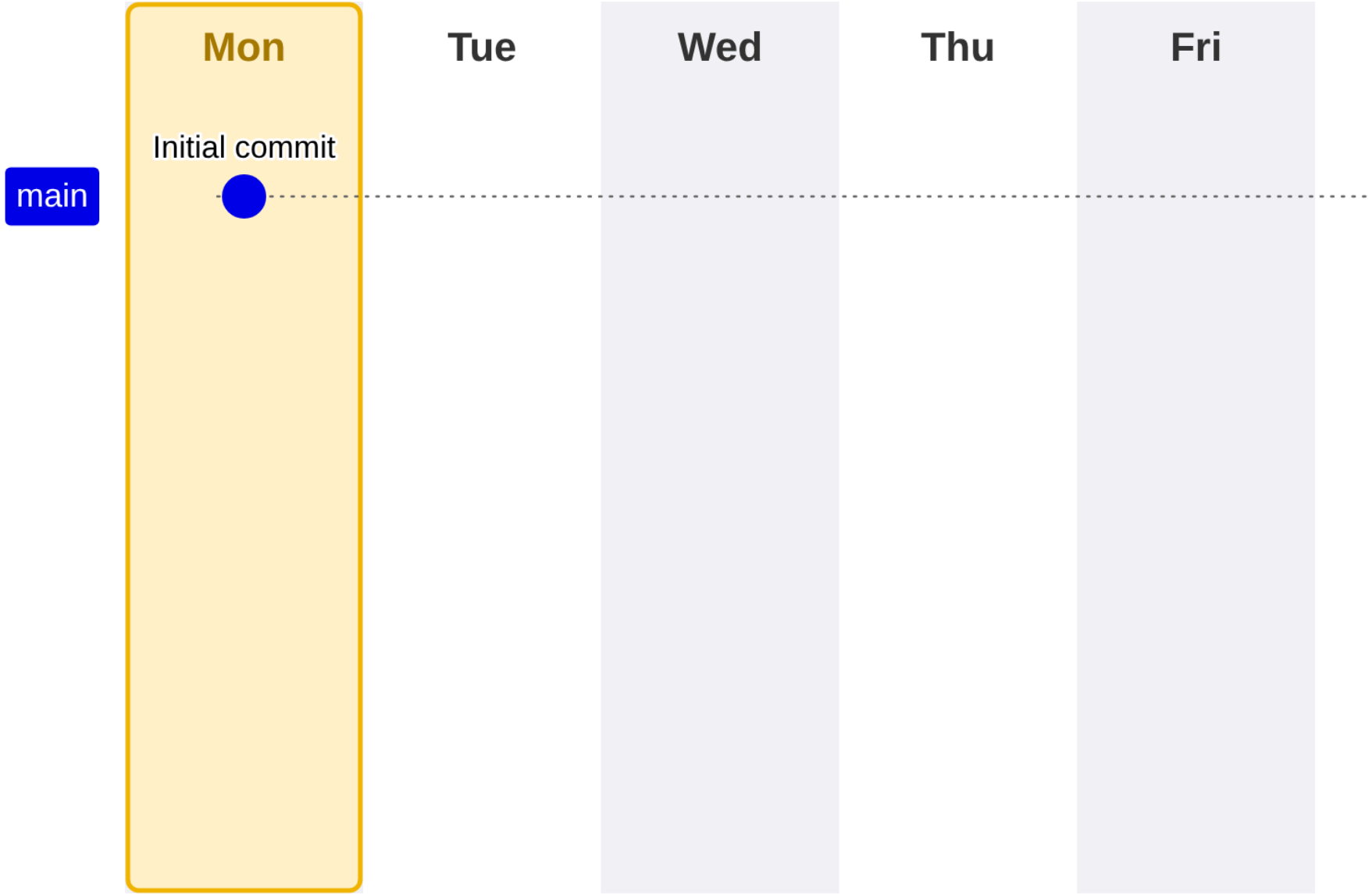
A **branch** is a separate line of changes in the *same* repository.

- `main` = the shared, stable line.
- `docs/add-reproducibility-note` = work in progress.
- Changes can be reviewed **before** being merged into `main`.



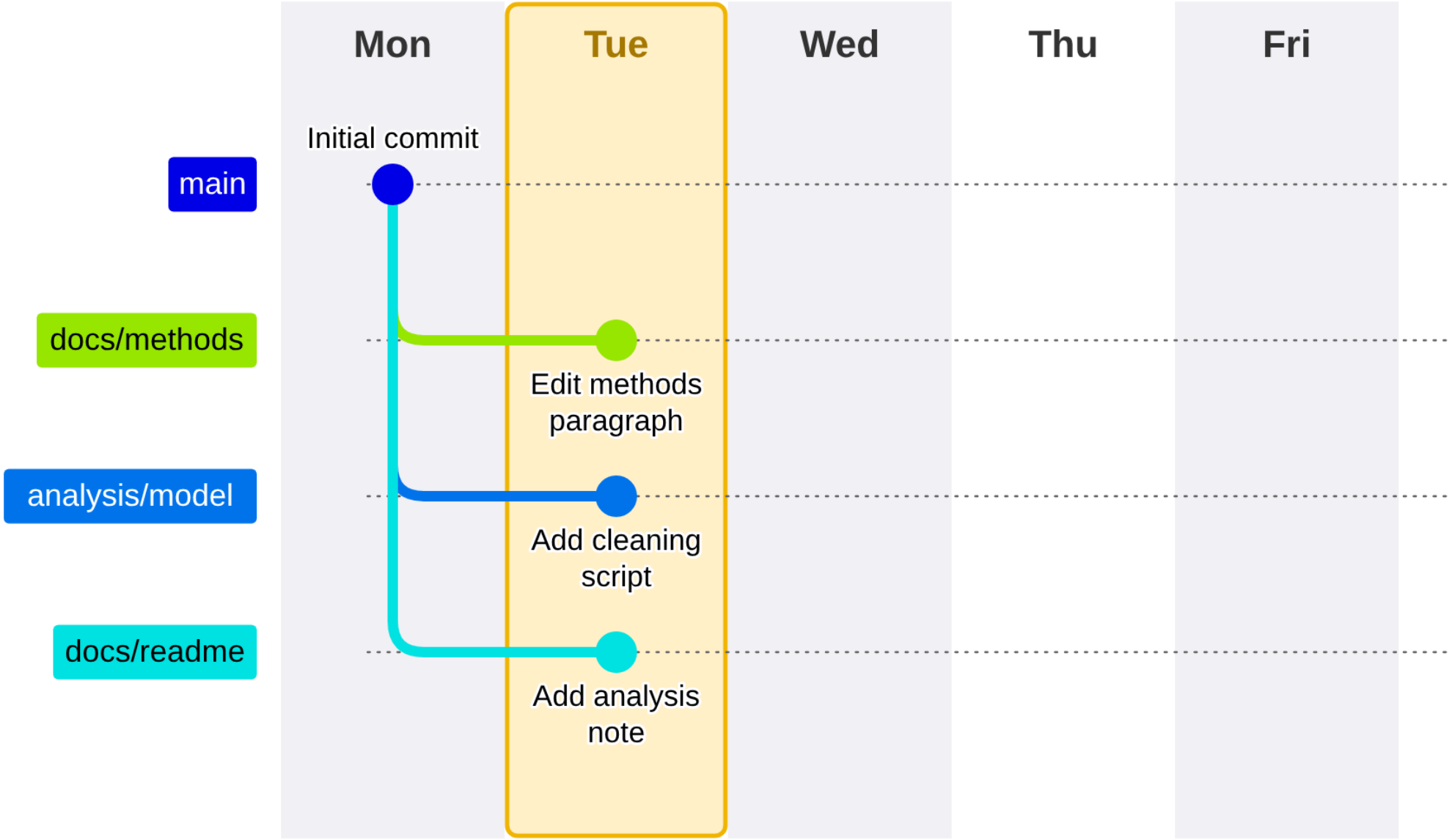
Branch ≠ separate repository.

One week workflow example



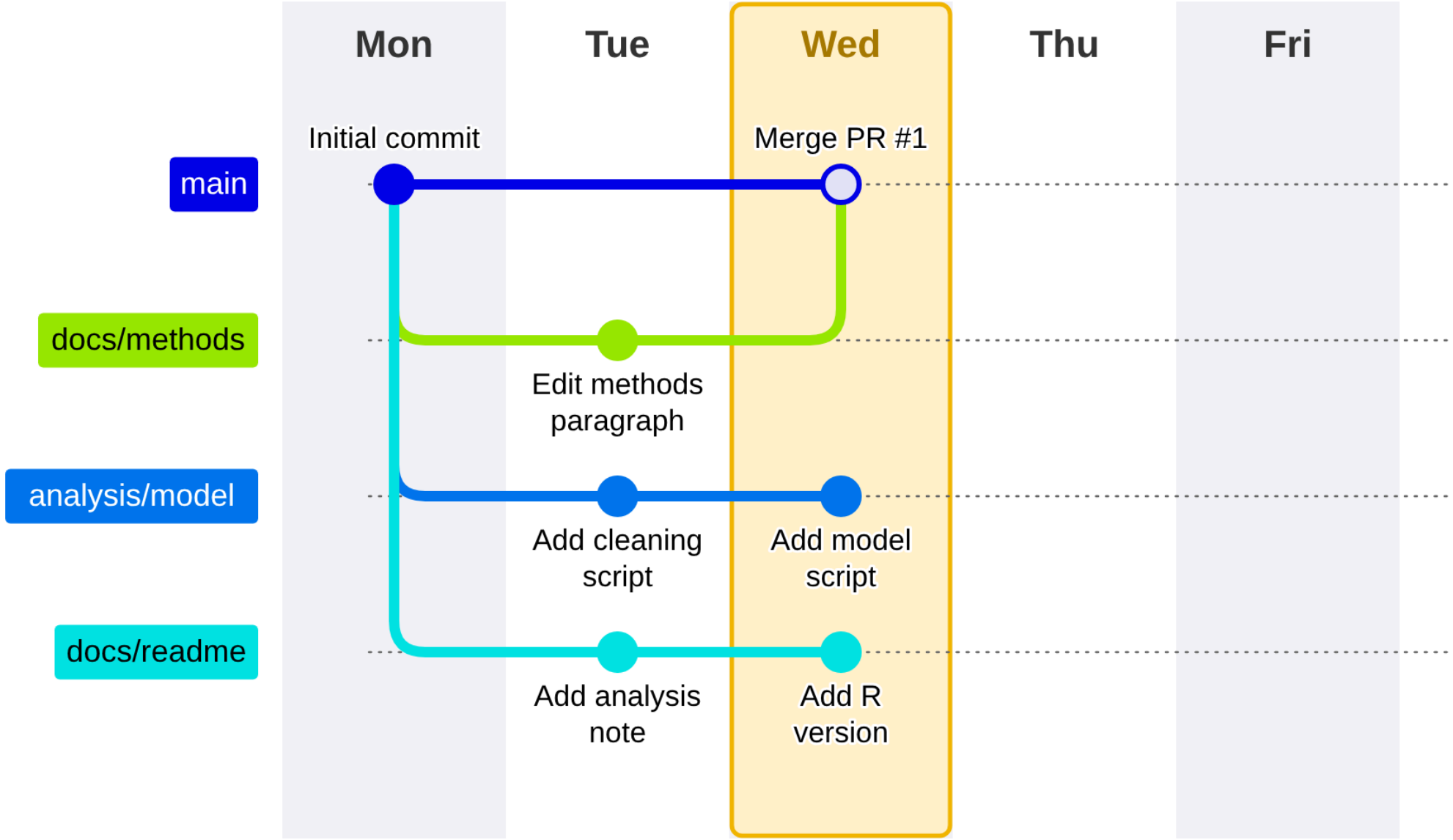
Project set up: first commit on main

One week workflow example



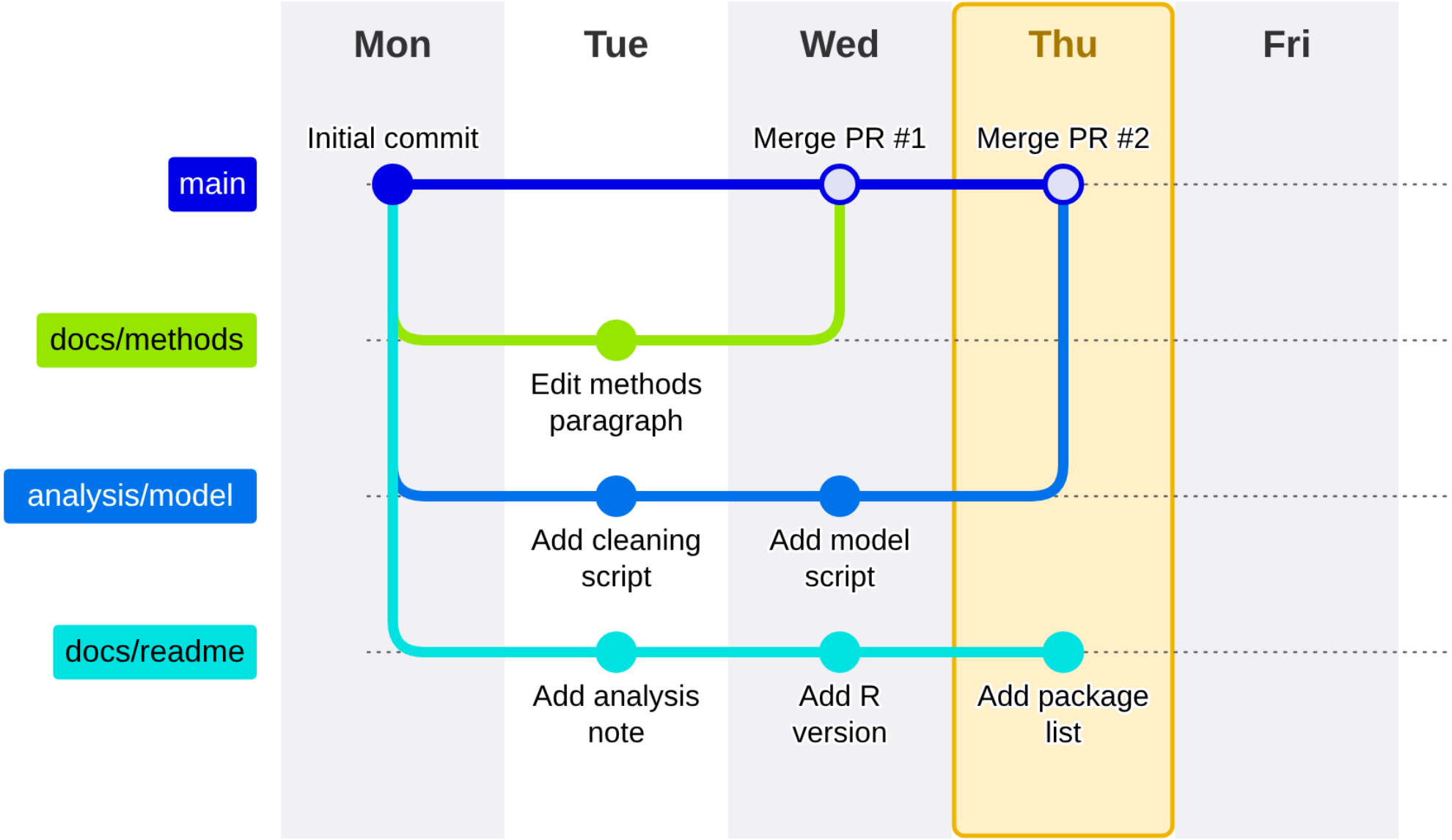
Work splits into three branches: methods, analysis and README

One week workflow example



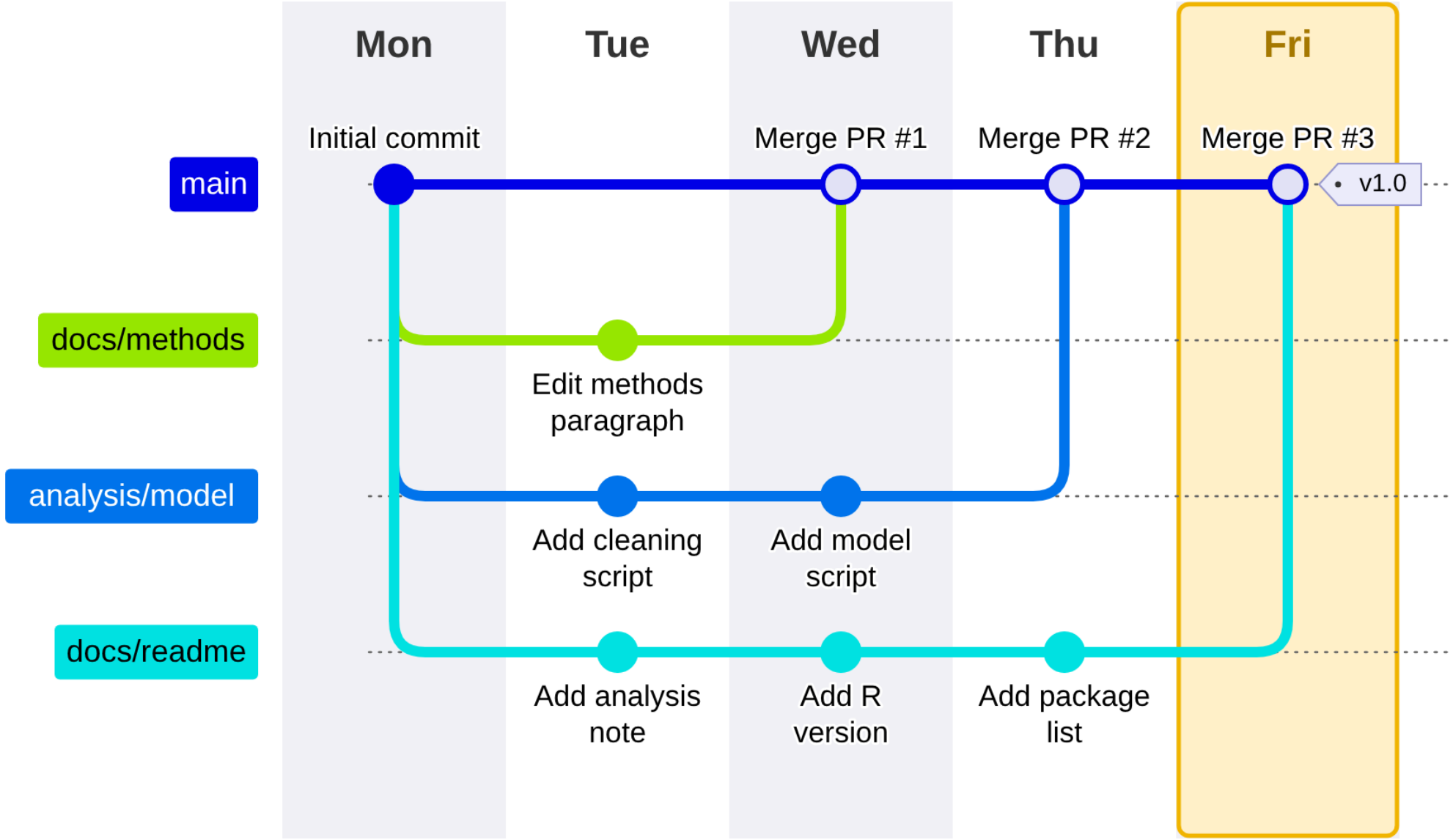
Methods edit reviewed and merged (PR #1); the other branches continue

One week workflow example

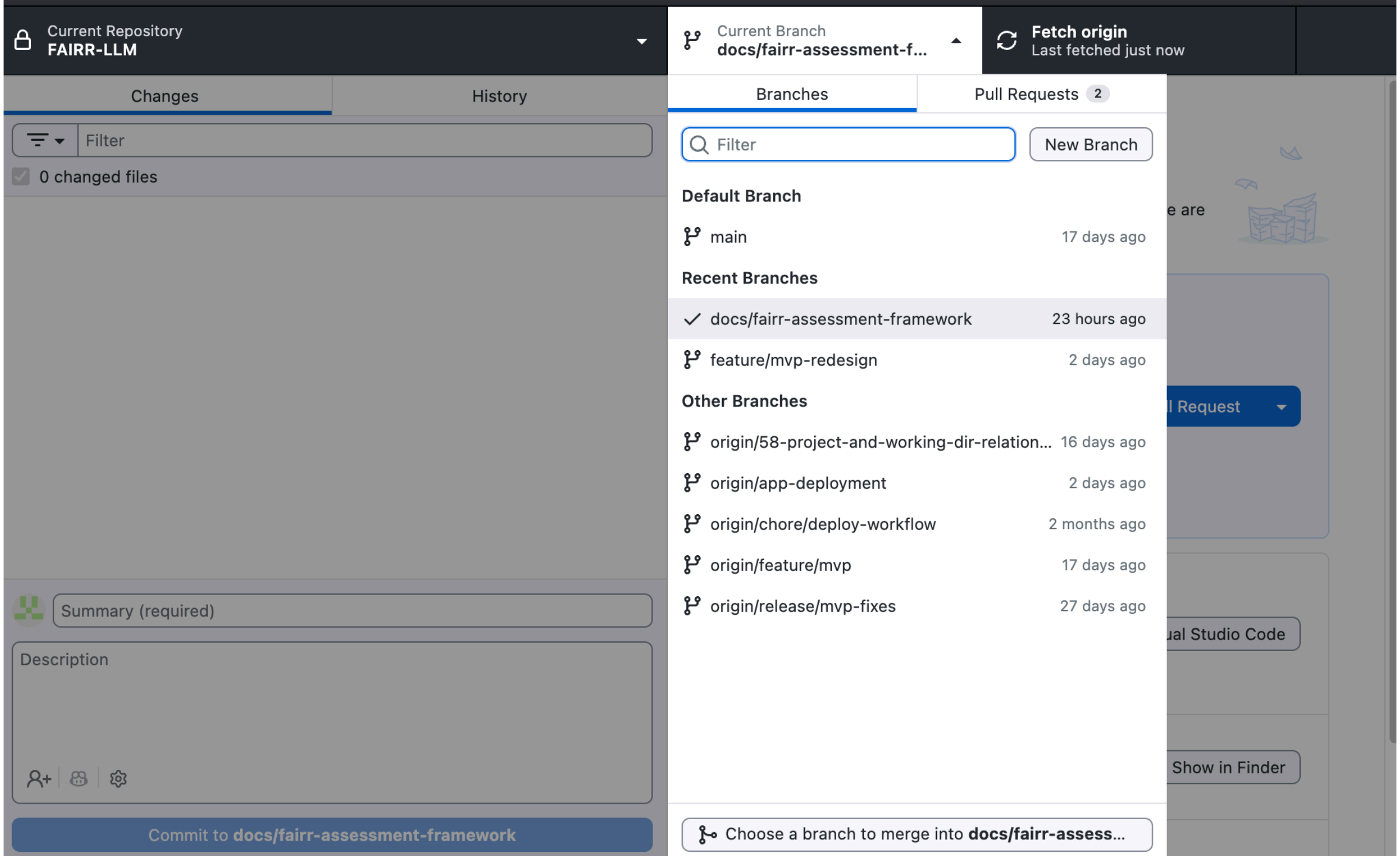


Analysis reviewed and merged (PR #2)

One week workflow example



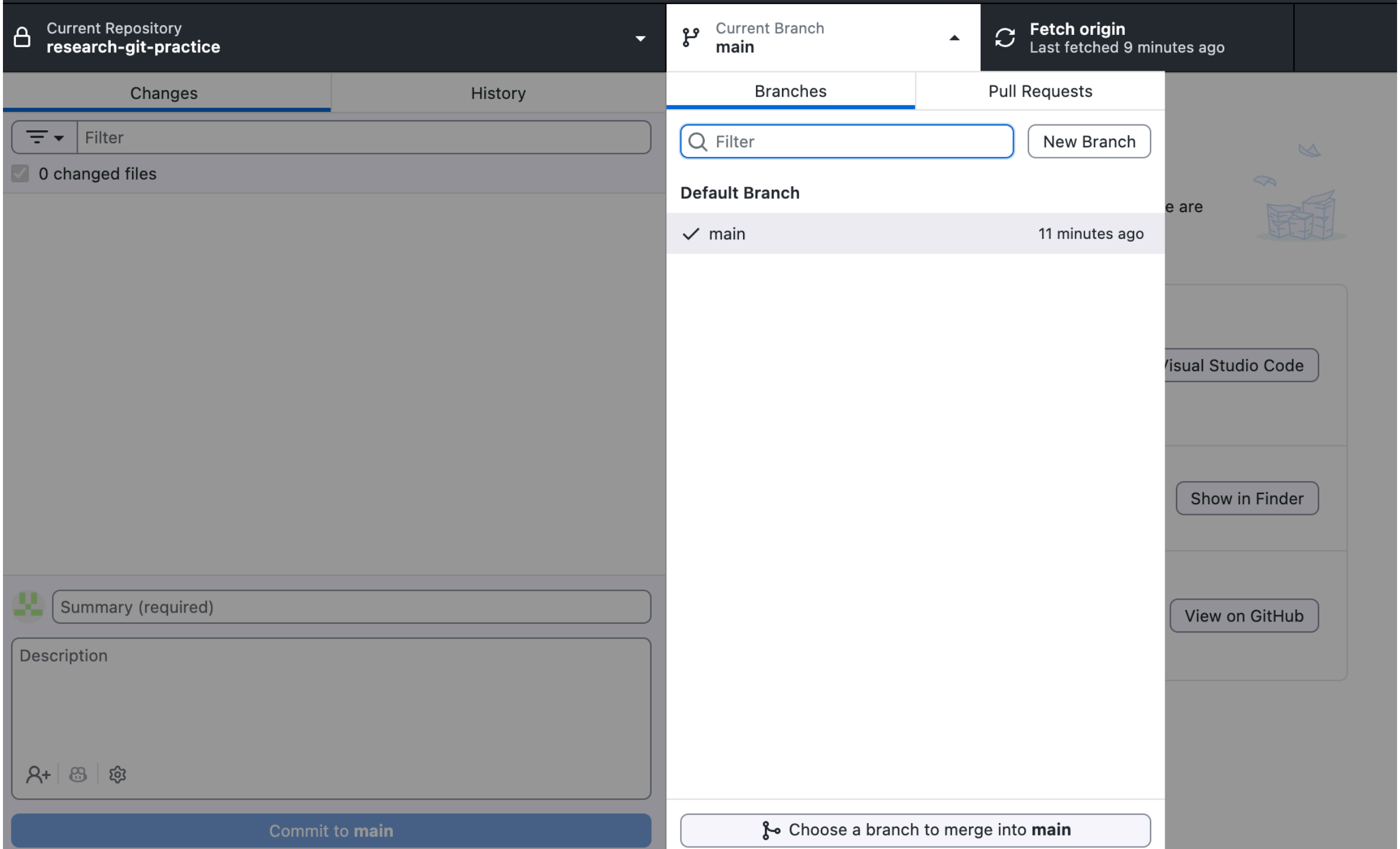
README notes merged (PR #3); main tagged v1.0 so others can cite this exact version


Current Repository
FAIRR-LLM
Current Branch
docs/fairr-assessment-f...
Fetch origin
Last fetched just now
Changes
History
Filter
0 changed files
Summary (required)
Description
Commit to docs/fairr-assessment-framework
Branches
Pull Requests 2
Filter
New Branch
Default Branch
main 17 days ago
Recent Branches
docs/fairr-assessment-framework 23 hours ago
feature/mvp-redesign 2 days ago
Other Branches
origin/58-project-and-working-dir-relation... 16 days ago
origin/app-deployment 2 days ago
origin/chore/deploy-workflow 2 months ago
origin/feature/mvp 17 days ago
origin/release/mvp-fixes 27 days ago
Choose a branch to merge into docs/fairr-assess...
e are
l Request
ual Studio Code
Show in Finder

Branching in GitHub Desktop

1. Start from an up-to-date **main** branch.
2. Click **Current Branch → New Branch**.
3. Give it a descriptive name.
4. Edit, review, and **commit** as usual.
5. **Publish branch** to share it on GitHub.

Switching branches changes which version of your project you are working on.


Current Repository
research-git-practice
Current Branch
main
Fetch origin
Last fetched 9 minutes ago
Changes
History
Filter
0 changed files
Summary (required)
Description
Commit to main
Branches
Pull Requests
Filter
New Branch
Default Branch
main
11 minutes ago
Choose a branch to merge into main
e are
Visual Studio Code
Show in Finder
View on GitHub

Exercise 6 · Work on a branch (10 min)

1. In GitHub Desktop, select `main`.
2. Create `docs/reproducibility-note`.
3. Open `docs/methods.md` and add one sentence about documentation.

> The project uses the SoDa SORTD datasets.

4. Review the diff and **commit**.
5. Click **Publish branch**.
6. On GitHub, verify both branches exist.

Quiz

Does `main` contain your branch’s new sentence yet?

Quiz · Answer

No. Your change is on the new branch.

- A branch lets you develop a proposed change independently.
- The change is **not yet in** `main`.
- The branch is now visible on GitHub because you **published** it.

How does a colleague decide whether to incorporate it?

> Next: an issue and a pull request.

Two GitHub collaboration tools

Issue = work to be done

> “Document which observations are excluded from the analysis.”

Pull request (PR) = proposed change to review and merge

> “I updated `docs/methods.md` to clarify exclusion rules.”

Typical sequence: Issue → branch → commits → PR → review → merge.

Research collaboration scenario

Person A = repository owner

- Opens a GitHub issue.
- Shares access with Person B to their `research-git-practice` repository.

Person B = contributor / author

- Clones A's repository.
- Makes a change on a **new branch**.
- Opens a pull request.

Person A = reviewer

- Reviews the proposal and merges it.

Both: Update the local `main` after the merge.

Create new issue

Add a title *

Document which observations are excluded from the analysis.

Add a description

Write Preview

Type your description here...

Create more Cancel Create ⌘ ⏎

Assignees

No one - Assign yourself

Labels

No labels

Projects

No projects

Milestone

No milestone

Issues
Assigned to me
Created by me
Mentioned
Recent activity

Views
Projects
Milestones
Labels

Feedback Preview
Collapse sidebar

Issues

New issue

is:issue state:open

Open 1 Closed 0 Author Labels Projects Milestones

Document which observations are excluded from the analysis.

fqixiang/research-git-practice#1 · fqixiang opened 1m ago

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also compare across forks. Learn more about diff comparisons here.

base: main ← compare: docs/reproducibility-note ✓ Able to merge. These branches can be automatically merged.

Add a title *

Docs/reproducibility note

Add a description

Write | Preview

Fix #1.

This pull request adds a new section to the `docs/methods.md` file outlining the data exclusion rules for the analysis. The section specifies criteria for excluding publications based on citation count, publication date, and peer review status.

Markdown is supported | Paste, drop, or click to add files

Create pull request

Reviewers

Copilot Balanced — Request

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use Closing keywords in the description to automatically close issues

Docs/reproducibility note #2

Merge status

Open fqixiang wants to merge 3 commits into `main` from `docs/reproducibility-note`

Conversation 0 | Commits 3 | Checks 0 | Files changed 1

fqixiang commented 1 minute ago

Fix #1.

This pull request adds a new section to the `docs/methods.md` file outlining the data exclusion rules for the analysis. The section specifies criteria for excluding publications based on citation count, publication date, and peer review status.

Mention @copilot in a comment to make changes to this pull request.

fqixiang added 3 commits 5 minutes ago

- docs: add analytic method 6eade17
- Revert "docs: add analytic method" ... 9f15eda
- docs: add data exclusion rules 60ef0d0

Reviewers

Copilot Balanced

Still in progress? Convert to draft

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may clo

Document which observations are excluded from the analysis. #1

Closed

fqixiang opened 12m ago

No description provided.

Create sub-issue Add artifact

fqixiang mentioned this 2m ago

Docs/reproducibility note #2

fqixiang closed this as completed in #2 1m ago

Exercise 7A · Share and plan (5 min)

Person A:

1. On your repository: **Settings → Collaborators → Add people**.
2. Invite Person B's GitHub username.

Person B:

3. Accept the invitation, then **clone A's repository**.

Person A:

4. Open a GitHub issue:

> “Add a sentence explaining what statistical methods are used in the analysis.”

5. Assign the issue to Person B.

Exercise 7B · Propose a change (10 min)

Person B:

1. On A's cloned repository, start from **main**.
2. Create branch `docs/clarify-methods`.
3. Add a sentence to `docs/methods.md`.

> "The project uses linear regression."

4. Review the diff → **Commit** → **Publish branch**.
5. Click **Preview/Create Pull Request** in Desktop.
6. On GitHub, open a PR **into** `main`; mention the issue ("Fix #1").

Do not merge your own PR yet.

Exercise 7C · Review, merge, synchronize (5 min)

Person A:

1. On GitHub, open B's PR → **Files changed**.
2. Leave a comment or **Approve** the change.
3. **Merge pull request** into `main`.

Both people:

4. In GitHub Desktop, **switch to `main`**.
5. **Fetch origin → Pull origin** (when offered).
6. Confirm the new sentence appears locally.

What happened?

Person B did **not** overwrite A's work.

Instead, B:

1. Proposed a change on a branch.
2. Made the diff visible in a pull request.
3. Received review.
4. Had the change merged into `main`.

What should a reviewer check?

Before merging research code or documentation:

- **Purpose:** What is this change supposed to achieve?
- **Scope:** Were only the intended files edited?
- **Diff:** What lines were added or removed?
- **Evidence:** If code changed, was it run or tested?
- **Risks:** Any confidential data, passwords, or unintended outputs?

> A green “Merge” button is not proof of correctness.

Quiz

What can a pull request show that an emailed file cannot?

Quiz · Answer

A pull request connects the proposed **diff** with:

- The reason for the change and its issue.
- Comments and review decisions.
- The commits and the eventual merge into `main`.

> **Both collaborators then pull the shared version.**

Daily workflow for a research project

```
PULL latest main
      ↓
CREATE/CHOSE a issue that you plan to work on
      ↓
CREATE a branch for one coherent change
      ↓
EDIT → REVIEW DIFF → COMMIT
      ↓
PUSH / PUBLISH branch
      ↓
OPEN PR → DISCUSS / REVIEW → MERGE
      ↓
PULL updated main
```

You now know the essentials of this workflow.

The five things to remember

1. **Commit**: record a meaningful local change.
2. **Push / Pull**: synchronize with GitHub.
3. **Branch**: keep new work separate from `main`.
4. **Pull request**: propose and review a change.
5. **History / Revert**: understand and recover prior work.

> **Git supports reproducibility; it does not guarantee it.**

Supercharge your research workflow

- AI-assisted coding
- AI-assisted writing

Message (⌘Enter to commit on "c...

Commit

Changes 37

.DS_Store M

Graph Auto

Add publisher example PDF for Patterns ...
Add unmodified Patterns LaTeX template...
Add original Patterns LaTeX template arc...
Create FINAL_SUBMISSION_PLAN.md Q...
Preserve complete colorful R1 snapshot ...
Create COVER_LETTER.te... origin/main
Update exact quotes Qixiang Fang
REBUTTAL.tex M
Final linguistic and structural pass of the ...
main.tex source_files M
Update main text quotes Qixiang Fang
Include more classical measurement-err...
Update measurement-error framing and ...
Refresh annotation-error citations Qixian...
Clarify annotation-error discussion in Se...
Did a whole pass over the rebuttal letter, ...
Merge overleaf-2026-09-29-1418 into ...
Updates from Overleaf Qixiang Fang
Redifferentiate rebuttal comments fro...
Convert rebuttal to LaTeX Qixiang Fang
Fix wording Qixiang Fang
Refine rebuttal Qixiang Fang
Clarify LLM measurement framing Qixian...
Revise rebuttal to address reviewers Qixi...
Add agreement metrics interpretation gu...
Add citation on iterative codebook devel...
Clarify multi-rater gold standard guidanc...
Minor linguistic changes based on Javier...
In the rebuttal, replaced ">" of direct qu...

```
468
469 \begin{quote}
470
471 If fine-tuning or another form of model adaptation is used,
    the examples used to update model parameters are
    additionally referred to as \emph{training data}. These
    training/adaptation examples may be drawn from the Explore
    subset, but should not be drawn from the Select or Evaluate
    subsets.
472
473 \end{quote}
474
    19 hidden lines
494
495 \begin{quote}
496
497 In LLM-based annotation, the prompt should not be invented
    independently from the research design. It should be
    derived from the codebook or rubric developed in Section 3,
    because that document defines the construct, the admissible
    labels, the decision rules, and the edge cases.
498
499 \end{quote}
500
501 We added the promptbook literature and expanded the
    discussion of
502 automated assistance. The revised manuscript makes their
    different purposes explicit:
503
504 \begin{quote}
505
506 Recent work describes this translation as from codebook to
    ``promptbook'': a reproducible set of prompts, examples,
    output constraints, model settings, and documentation that
    operationalizes the codebook for machine annotation
    (Stuhler et al., 2025; Törnberg, 2024).
507
508 \end{quote}
509
    208 hidden lines
718
719 \begin{quote}
720
```

```
468
469 \begin{quote}
470
471 If fine-tuning or another form of model adaptation is used,
    the labelled examples used to update model parameters are
    additionally referred to as \emph{training data}. These
    training/adaptation examples may be drawn from the Explore
    subset, but should not be drawn from the Select or Evaluate
    subsets.
472
473 \end{quote}
474
    19 hidden lines
494
495 \begin{quote}
496
497 In this process, the prompt should not be invented
    independently from the research design. Instead, it should
    be derived from the codebook or rubric developed in Section
    3, which defines the construct, the admissible labels, the
    decision rules, and the edge cases.
498
499 \end{quote}
500
501 We added the promptbook literature and expanded the
    discussion of
502 automated assistance. The revised manuscript makes their
    different purposes explicit:
503
504 \begin{quote}
505
506 Recent work describes this translation as \emph{from
    codebook to ``promptbook''}: a reproducible set of prompts,
    examples, output constraints, model settings, and
    documentation that operationalizes the codebook for machine
    annotation (Stuhler et al., 2025; Törnberg, 2024).
507
508 \end{quote}
509
    208 hidden lines
718
719 \begin{quote}
720
```

AI-assisted changes in general

If an AI assistant makes any change to your file(s), you should:

- **Make a plan**: what is the intended change?
- **Branch first** so the proposal is isolated.
- **Review the diff**: which files and lines changed?
- **Test / inspect outputs** before accepting the result.
- **Commit meaningful units** rather than a giant opaque change.

> Git records changes; **you** remain responsible for research validity.

Git with your coding tools

You learned Git through GitHub Desktop.

The same repository can also be managed using:

- VS Code's built-in Source Control interface
- RStudio's Git pane
- Git in a terminal

Same concepts: repo, commit, branch, push, pull, merge.

Writing research papers with Git

The same workflow can version-control:

- LaTeX source (`.tex`) and bibliographies (`.bib`)
- Manuscript figures and tables generated by code
- Revisions, comments, and writing contributions

Can be especially useful for:

- **Git + AI integration**
- **Overleaf + Git integration**

File Edit Insert View Format Help patterns_revision

Integrations

GitHub
Sync with a GitHub repository.

Git
Git clone this project.

Dropbox
Sync all projects with Dropbox.

ReadCube
Cite directly or import references

Zotero
Cite directly or import references

Mendeley
Cite directly or import references

main.tex × REBUTTAL.tex numbered.bst numcompress.sty

Recompile

Code Visual

```
33
34  % Hyperlinks
35  \usepackage{hyperref}
36
37  \renewcommand{\sectionautorefname}{Section}
38  \renewcommand{\subsectionautorefname}{Section}
39
40  \linenumbers
41
42  % Highlight palette
43  \definecolor{task}{RGB}{222,235,247}
44  \definecolor{context}{RGB}{253,219,199}
45  \definecolor{input}{RGB}{213,232,212}
46  \definecolor{output}{RGB}{249,237,187}
47
48  % Box styling
49  \tcbset{
50    enhanced,
51    sharpish corners,
52    before skip=10pt, after skip=10pt,
53    boxsep=1em, left=1em, right=1em, top=0.8em, bottom=0.8em
54  }
55
```

Useful official guides

- Getting started with GitHub Desktop
- Cloning a repository
- Creating a PR from Desktop
- Inviting collaborators
- Reviewing pull requests
- Reverting commits
- Carpentries: GitHub without the command line

Optional · When Git gets complicated

Merge conflict = two sets of changes cannot be reconciled automatically.

Common prevention habits:

- Pull before starting new work.
- Keep branches and pull requests focused.
- Coordinate when editing the same lines.
- Review conflicts carefully; never choose an option blindly.

There is no need to master conflict resolution today.

What do the Git commands look like?

GitHub Desktop action	Command-line equivalent
Clone	`git clone URL`
Review status	`git status`
Record a change	`git add …` then `git commit -m "…"`
Send changes	`git push`
Retrieve/merge remote changes	`git pull`
Create and switch branch	`git switch -c branch-name`

These are optional ways to perform the same tasks.